

课程笔记

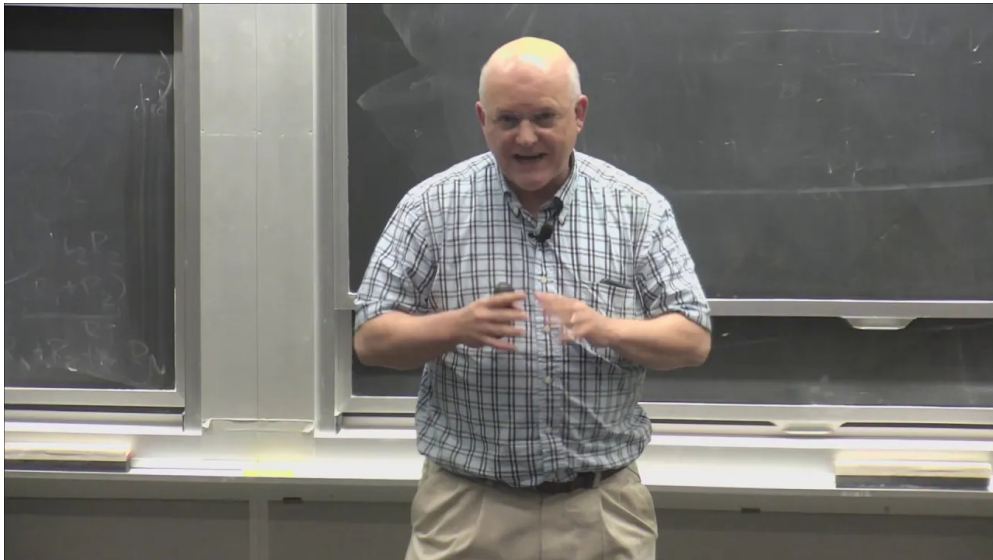
Introduction and Matrix Multiplication

MIT 6.172 Performance Engineering of Software Systems, Fall 2018, Lecture 1

讲者: Charles Leiserson (MIT)

lecture-to-notes

2026 年 3 月 26 日



视频作者/频道: MIT OpenCourseWare

发布日期: Fall 2018

视频时长: 60 分钟

视频链接: https://www.youtube.com/watch?v=o7h_sYmk_oc

目录

1 引言：性能是货币	2
1.1 本章小结	2
2 实验平台与理论峰值	2
2.1 本章小结	3
3 从 Python 到 C：语言选择的影响	3
3.1 本章小结	4
4 循环交换与 Cache 行为	4
4.1 编译器优化标志	5
4.2 本章小结	6
5 并行化与分块	6
5.1 Tiled Matrix Multiplication	6
5.2 递归并行	7
5.3 本章小结	7
6 最终成绩：53,000× 加速	8
6.1 本章小结	9
7 总结与延伸	9
7.1 讲者的核心信息	9
7.2 结构化提炼	9
7.3 开放问题	9
7.4 拓展阅读	10

1 引言：性能是货币

Charles Leiserson 是 MIT 的传奇教授——《算法导论》（CLRS）的合著者、Cilk 并行编程框架的发明者。这门课的核心问题：**如何让软件跑得更快？**

Performance is Currency

性能不仅仅是“更快”——它是一种**货币**。你可以用性能来“购买”：

- 更好的用户体验（响应时间）
- 更低的成本（更少的服务器）
- 更多的功能（在相同时间内做更多计算）
- 可行性（原本不可能的计算变得可能）

1.1 本章小结

性能工程不是“微优化”——它是在有限资源下获得最大能力的系统性方法。

2 实验平台与理论峰值

AWS c4.8xlarge Machine Specs	
Feature	Specification
Microarchitecture	Haswell (Intel Xeon E5-2666 v3)
Clock frequency	2.9 GHz
Processor chips	2
Processing cores	9 per processor chip
Hyperthreading	2 way
Floating-point unit	8 double-precision operations, including fused-multiply-add, per core per cycle
Cache-line size	64 B
L1-icache	32 KB private 8-way set associative
L1-dcache	32 KB private 8-way set associative
L2-cache	256 KB private 8-way set associative
L3-cache (LLC)	25 MB shared 20-way set associative
DRAM	60 GB
Peak = $(2.9 \times 10^9) \times 2 \times 9 \times 16 = 836 \text{ GFLOPS}$	
© 2008-2018 by the MIT 6.172 Lecturers	

图 1: AWS c4.8xlarge 机器规格：Haswell Xeon E5-2666 v3，理论峰值 836 GFLOPS¹

¹视频画面时间区间：00:18:15–00:18:45。帧实际内容：slide “AWS c4.8xlarge Machine Specs”，列出处理器频率 2.9 GHz、2 芯片 × 9 核、L1 32KB、L2 256KB、L3 25MB、DRAM 60GB。Peak = $(2.9 \times 10^9) \times 2 \times 9 \times 16 = 836 \text{ GFLOPS}$ 。

理论峰值的计算

$$\begin{aligned}\text{Peak} &= f_{\text{clock}} \times N_{\text{chips}} \times N_{\text{cores}} \times \text{FLOPs/cycle} \\ &= 2.9 \times 10^9 \times 2 \times 9 \times 16 = 836 \text{ GFLOPS}\end{aligned}$$

- $f_{\text{clock}} = 2.9 \text{ GHz}$
- $N_{\text{chips}} = 2, N_{\text{cores}} = 9/\text{chip}$
- $\text{FLOPs/cycle} = 16$ (双精度 FMA: 8 个乘加 $\times 2$)

整门课的目标：看看矩阵乘法能逼近这个峰值多少。

2.1 本章小结

理论峰值 836 GFLOPS 是衡量优化效果的标尺。矩阵乘法是完美的 benchmark——计算密集、可并行、有已知最优算法。

3 从 Python 到 C：语言选择的影响

Version 3: C

```
#include <stdlib.h>
#include <stdio.h>
#include <sys/time.h>

#define n 4096
double A[n][n];
double B[n][n];
double C[n][n];

float tdiff(struct timeval *start,
            struct timeval *end) {
    return (end->tv_sec-start->tv_sec) +
        1e-6*(end->tv_usec-start->tv_usec);
}

int main(int argc, const char *argv[]) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            A[i][j] = (double)rand() / (double)RAND_MAX;
            B[i][j] = (double)rand() / (double)RAND_MAX;
            C[i][j] = 0;
        }
    }

    struct timeval start, end;
    gettimeofday(&start, NULL);

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            for (int k = 0; k < n; ++k) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    gettimeofday(&end, NULL);
    printf("%0.6f\n", tdiff(&start, &end));
    return 0;
}
```

Using the Clang/LLVM 5.0 compiler
Running time = 1,156 seconds
 ≈ 19 minutes,
or about $2\times$ faster than Java and
about $18\times$ faster than Python.

```
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
        for (int k = 0; k < n; ++k) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
```

© 2008–2018 by the MIT 6.172 Lecturers 26

图 2: Version 3: C 语言实现, Clang/LLVM 5.0 编译, 1156 秒, 比 Java 快 $2\times$, 比 Python 快 $18\times^3$

```
1 for (int i = 0; i < n; ++i)
2     for (int j = 0; j < n; ++j)
```

³视频画面时间区间: 00:24:30–00:25:00。帧实际内容: slide “Version 3: C”, 展示 C 语言矩阵乘法代码 (三重循环 i,j,k), Running time = 1,156 seconds ≈ 19 minutes。

```

3     for (int k = 0; k < n; ++k)
4         C[i][j] += A[i][k] * B[k][j];

```

Listing 1: Version 3: 最基本的 C 矩阵乘法

Leiserson 展示了矩阵乘法的逐步优化，从 Python 开始：

Version	Implementation	Time (s)	Speedup	% Peak
1	Python	21041.67	1×	0.001
2	Java	2387.32	9×	0.007
3	C	1155.77	18×	0.014

Python 慢 18

Python 慢是因为它是**解释执行 + 动态类型**。每次 `C[i][j] += ...` 都要做类型检查、方法查找、对象分配。C 直接操作内存——没有中间层。

但这并不意味着你不应该用 Python——开发效率和运行效率是不同的“货币”。

3.1 本章小结

语言选择带来 18x 差异 (Python → C)，但这只达到了峰值的 0.014%。真正的性能提升来自后续的算法和硬件层优化。

4 循环交换与 Cache 行为

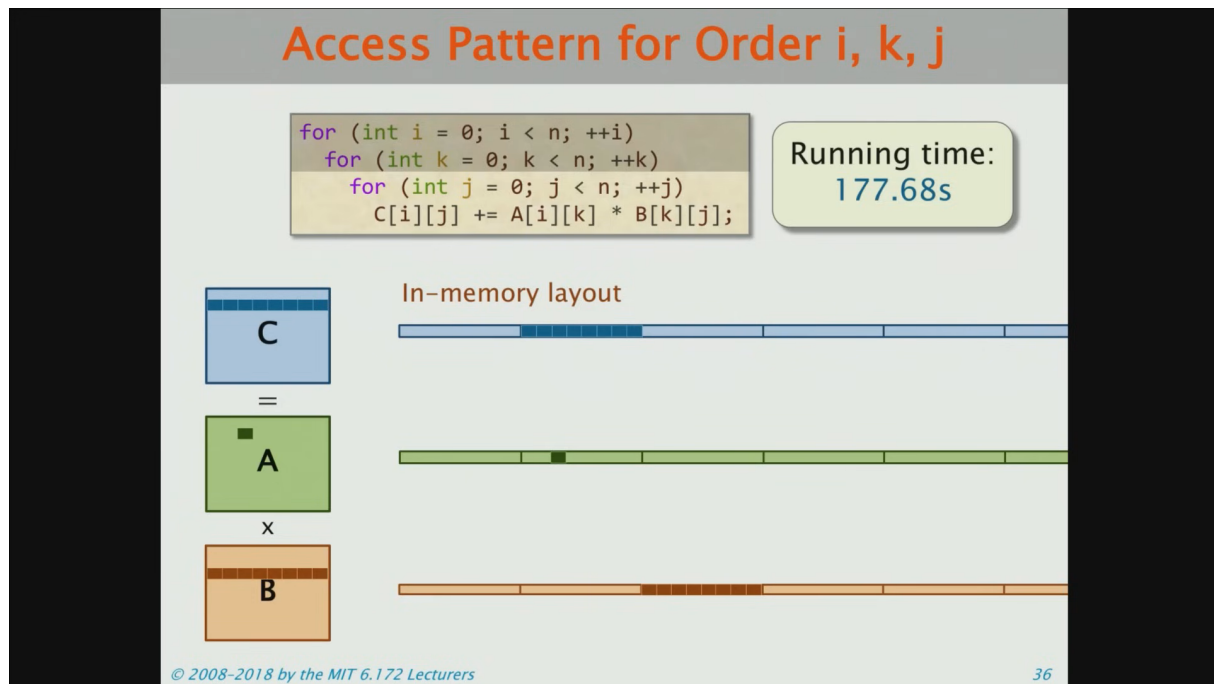


图 3: 循环顺序 i,k,j 的内存访问模式：Cache-line 级别的可视化⁴

循环交换 (Loop Interchange) 的威力

将 i, j, k 循环改为 i, k, j 顺序:

$$1155.77\text{s} \rightarrow 177.68\text{s} \quad (6.5 \times \text{speedup})$$

原因: i, k, j 顺序下, 最内层循环 j 遍历 $C[i][j]$ 和 $B[k][j]$ ——都是连续内存访问 (row-major)。而 i, j, k 顺序下, $B[k][j]$ 每次跳一整行——cache miss。

4.1 编译器优化标志

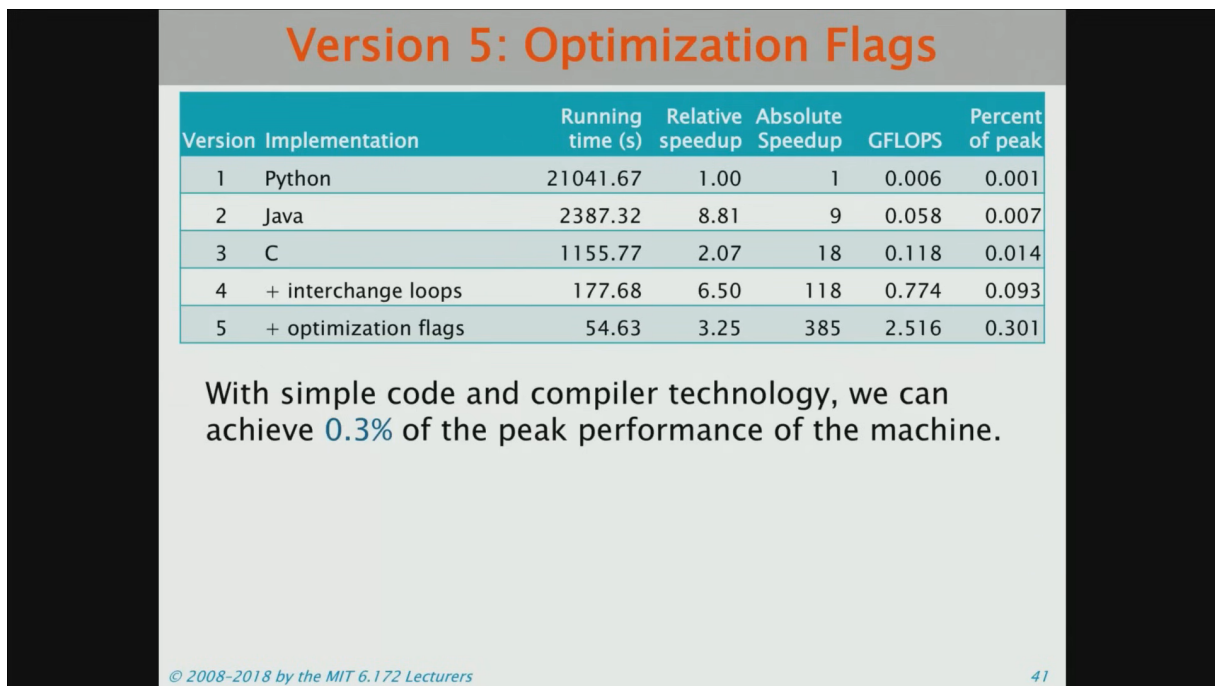


图 4: Version 5: 加 -O3 优化标志, 54.63s, 达到峰值的 0.3%⁵

编译器能做什么

-O3 启用的关键优化:

- 循环展开 (Loop Unrolling)
- 向量化 (Auto-Vectorization)
- 指令调度 (Instruction Scheduling)
- 公共子表达式消除 (CSE)

但即使开了 -O3, 也只达到峰值的 0.3%——编译器无法自动做循环分块和并行化。

⁴视频画面时间区间: 00:33:15–00:33:45。帧实际内容: slide “Access Pattern for Order i, k, j”, 展示三重循环 i, k, j 顺序的代码和内存布局图。C/A/B 矩阵的行优先存储下的 cache line 访问模式。Running time: 177.68s。

⁵视频画面时间区间: 00:35:45–00:36:15。帧实际内容: slide “Version 5: Optimization Flags”, 完整表格显示 V1–V5 的时间和加速比。“With simple code and compiler technology, we can achieve 0.3% of the peak performance of the machine.”

4.2 本章小结

循环交换利用 cache 空间局部性，带来 6.5x 提升。编译器 -O3 再给 3.25x。但总共只到峰值的 0.3%——后面还有 300x 的空间。

5 并行化与分块

5.1 Tiled Matrix Multiplication

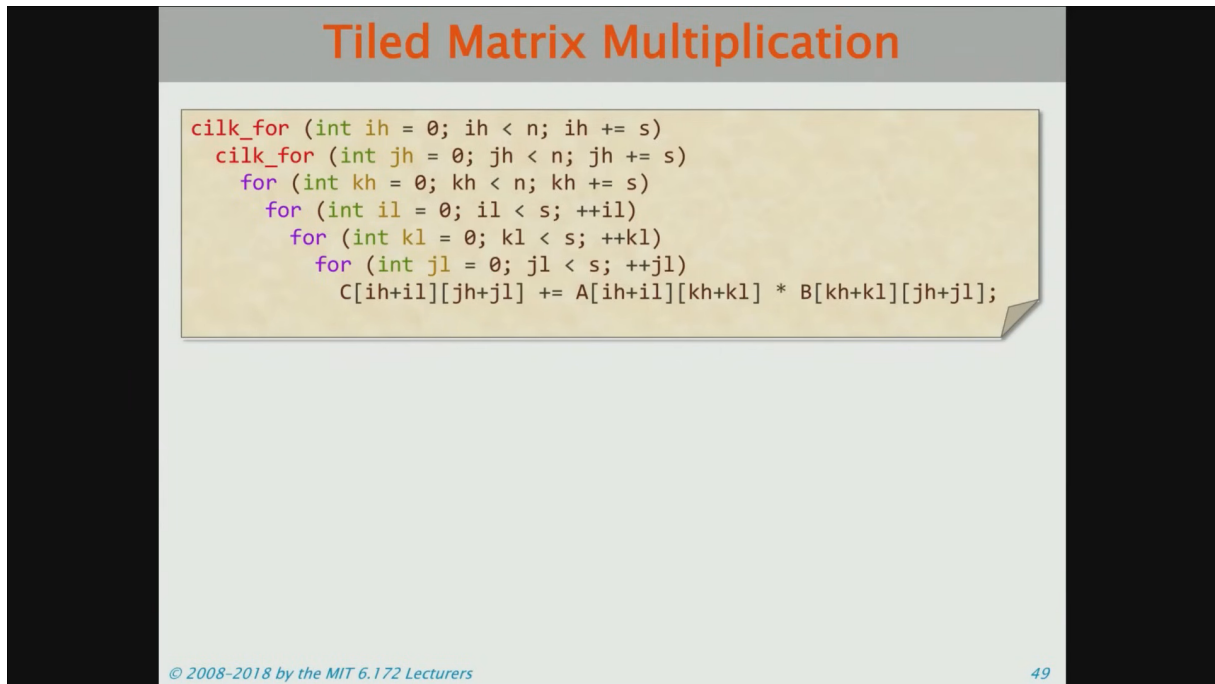


图 5: Tiled（分块）矩阵乘法：cilk_for 并行 + 6 重循环⁶

```
1 cilk_for (int ih = 0; ih < n; ih += s)
2   cilk_for (int jh = 0; jh < n; jh += s)
3     for (int kh = 0; kh < n; kh += s)
4       for (int il = 0; il < s; ++il)
5         for (int kl = 0; kl < s; ++kl)
6           for (int jl = 0; jl < s; ++jl)
7             C[ih+il][jh+jl] += A[ih+il][kh+kl] * B[kh+kl][jh+jl];
```

Listing 2: Tiled 矩阵乘法（Cilk 并行）

⁶视频画面时间区间：00:43:15–00:43:45。帧实际内容：slide “Tiled Matrix Multiplication”，展示 cilk_for 并行的分块矩阵乘法代码（ih, jh, kh 为块索引，il, kl, jl 为块内索引）。

5.2 递归并行

Recursive Parallel Matrix Multiply

```
void mm_dac(double *restrict C, int n_C,
            double *restrict A, int n_A,
            double *restrict B, int n_B,
            int n)
{ // C += A * B
  assert((n & (-n)) == n);
  if (n <= 1)
    *C += *A * *B;
  else {
    #define X(M,r,c) (M + (r*(n_ ## M) + c*(n_ ## M)))
    cilk_spawn mm_dac(X(C,0,0), n_C, X(A,0,0), n_A, X(B,0,0), n_B, n/2);
    cilk_spawn mm_dac(X(C,0,1), n_C, X(A,0,0), n_A, X(B,0,1), n_B, n/2);
    cilk_spawn mm_dac(X(C,1,0), n_C, X(A,1,0), n_A, X(B,0,0), n_B, n/2);
    mm_dac(X(C,1,1), n_C, X(A,1,0), n_A, X(B,0,1), n_B, n/2);
    cilk_sync;
    cilk_spawn mm_dac(X(C,0,0), n_C, X(A,0,1), n_A, X(B,1,0), n_B, n/2);
    cilk_spawn mm_dac(X(C,0,1), n_C, X(A,0,1), n_A, X(B,1,1), n_B, n/2);
    cilk_spawn mm_dac(X(C,1,0), n_C, X(A,1,1), n_A, X(B,1,0), n_B, n/2);
    mm_dac(X(C,1,1), n_C, X(A,1,1), n_A, X(B,1,1), n_B, n/2);
    cilk_sync;
  }
}
```

The base case is too small. We must **coarsen** the recursion to overcome function-call overheads.

Running time: **93.93s**
... about **50×** slower than the last version!

© 2008–2018 by the MIT 6.172 Lecturers 56

图 6: 递归并行矩阵乘法: cilk_spawn 递归分治, 93.93s (比上一版慢 50×!) ⁷

递归并行反而慢了？

递归版本 (93.93s) 比并行分块版本 (1.79s) 慢了 50 倍！原因：**递归到单个元素**的 base case 太小，函数调用开销淹没了计算。

解决：**粗化递归** (Coarsen the recursion) —— 当子问题小于某个阈值时切换到迭代实现。

5.3 本章小结

并行化 + 分块带来了巨大提升 (18x → 6921x)。但递归并行的 base case 必须足够大，否则函数调用开销会反噬性能。

⁷ 视频画面时间区间：00:49:30–00:50:00。帧实际内容：slide “Recursive Parallel Matrix Multiply”，展示 cilk_spawn 递归代码。标注 “The base case is too small. We must coarsen the recursion to overcome function-call overheads.” Running time: 93.93s。

6 最终成绩：53,000× 加速

Version 10: AVX Intrinsic						
Version	Implementation	Running time (s)	Relative speedup	Absolute Speedup	GFLOPS	Percent of peak
1	Python	21041.67	1.00	1	0.006	0.001
2	Java	2387.32	8.81	9	0.058	0.007
3	C	1155.77	2.07	18	0.118	0.014
4	+ interchange loops	177.68	6.50	118	0.774	0.093
5	+ optimization flags	54.63	3.25	385	2.516	0.301
6	Parallel loops	3.04	17.97	6,921	45.211	5.408
7	+ tiling	1.79	1.70	11,772	76.782	9.184
8	Parallel divide-and-conquer	1.30	1.38	16,197	105.722	12.646
9	+ compiler vectorization	0.70	1.87	30,272	196.341	23.486
10	+ AVX intrinsics	0.39	1.76	53,292	352.408	41.677

© 2008–2018 by the MIT 6.172 Lecturers 68

图 7: 完整优化阶梯：从 Python 到 AVX Intrinsics, 53,292× 加速，达到峰值的 41.7%⁸

V	Implementation	Time (s)	Abs. Speedup	GFLOPS	% Peak
1	Python	21041.67	1	0.006	0.001
2	Java	2387.32	9	0.058	0.007
3	C	1155.77	18	0.118	0.014
4	+ interchange loops	177.68	118	0.774	0.093
5	+ optimization flags	54.63	385	2.516	0.301
6	Parallel loops	3.04	6,921	45.211	5.408
7	+ tiling	1.79	11,772	76.782	9.184
8	Parallel divide-and-conquer	1.30	16,197	105.722	12.646
9	+ compiler vectorization	0.70	30,272	196.341	23.486
10	+ AVX intrinsics	0.39	53,292	352.408	41.677

⁸视频画面时间区间：00:57:00–00:57:30。帧实际内容：slide “Version 10: AVX Intrinsics”，完整 10 版本对比表。Version 10: + AVX intrinsics, 0.39s, 53,292x speedup, 352.408 GFLOPS, 41.677% of peak。

10 个版本的优化层次

1. 语言选择: Python $\rightarrow$ C (18 $\times$)
2. 数据局部性: 循环交换 (6.5 $\times$)
3. 编译器优化: -O3 标志 (3.25 $\times$)
4. 并行化: Cilk 多核 (18 $\times$)
5. Cache 分块: Tiling (1.7 $\times$)
6. 分治算法: 递归 + 粗化 (1.4 $\times$)
7. 向量化: 编译器自动向量化 (1.9 $\times$)
8. 手写 SIMD: AVX intrinsics (1.8 $\times$)

从 0.001% 到 41.7% 峰值——每一层都不可或缺。

6.1 本章小结

53,000 倍加速来自 8 个不同层次的优化叠加。没有“银弹”——每一层贡献 2–18 倍。这就是性能工程的精髓：**系统性地消除每一层的瓶颈**。

7 总结与延伸

7.1 讲者的核心信息

$$\text{Python (0.001\%)} \xrightarrow{\times 53,292} \text{AVX (41.7\% peak)}$$

7.2 结构化提炼

四个核心 Takeaway

1. **Performance is Currency**——性能不是奢侈品，而是决定产品可行性的基础资源
2. **优化是分层的**——语言 $\rightarrow$ 算法 $\rightarrow$ 编译器 $\rightarrow$ 并行 $\rightarrow$ 缓存 $\rightarrow$ SIMD，每层独立贡献
3. **理解硬件是前提**——不知道 cache line 大小就无法做循环交换；不知道 FMA 就无法估算峰值
4. **53,000 $\times$ 不是极限**——41.7% 峰值说明还有 2.4 $\times$ 提升空间（Intel MKL 能到 >80%）

7.3 开放问题

1. 为什么 Intel MKL/BLAS 能达到 80%+ 峰值而手写代码只到 41.7%?
2. GPU 上的矩阵乘法优化策略有什么不同?
3. AI 框架（PyTorch/TensorFlow）的矩阵乘法用了哪些上述技术?

7.4 拓展阅读

- MIT 6.172 完整课程: <https://ocw.mit.edu/6-172F18>
- Leiserson et al. “There’s plenty of room at the Top” Science, 2020
- Intel Intrinsic Guide: <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/>